VIT CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

CLOUD INFRASTRUCTURE AND ARCHITECTURE

DIGITAL ASSIGNMENT 3 REPORT

CAMPUSFIND

Cloud-Based Campus Lost & Found System — High-Availability Scalability & Security Architecture

Academic Detail Information	Specification
Student Name	Sabarish Vishwanath Raja
Register Number	25BCE1230
Course	Cloud Infrastructure and Architecture
Course Code	BACSE344
Faculty	Dr. P. Anandan
Slot	C1
Submission Date	30 October 2026
Production URL	https://campusfind.vercel.app
Source Code Repository	https://github.com/SabarishVishwanathRaja/campusfind/tree/assignment-3

LIVE APPLICATION

https://campusfind.vercel.app

Submitted in partial fulfillment of the course requirements for BACSE344 — Cloud Infrastructure and Architecture

Abstract

CampusFind is a cloud-native campus belongings recovery platform engineered to address the critical logistical challenge of managing lost, found, and misplaced personal items across high-density educational campus environments. Building upon the foundational three-tier cloud deployment established in Digital Assignment 1, this report documents the advanced enterprise cloud engineering implemented for Digital Assignment 3, focusing on **horizontal scalability, high-availability reverse proxy load balancing, fault tolerance under chaos failure injection, multi-factor authentication (TOTP), role-based access control (RBAC), and container security hardening with Aqua Security Trivy**.

The architecture decouples the presentation layer (React 18 on Vercel's Edge CDN) from an active backend microservice cluster managed via Docker Compose behind an Nginx reverse proxy load balancer, backed by a serverless relational database on Neon PostgreSQL with Cloudinary multimedia object storage. Empirical stress testing demonstrates significant throughput improvements (+124.9%) and zero-downtime failover capabilities during node terminations. Container security hardening reduces image footprint by 85.1% and eliminates 100% of Critical vulnerabilities, ensuring alignment with modern data protection regulations including India's DPDP Act 2023 and GDPR.

1. Introduction and Operational Problem Analysis

1.1 Background & Operational Context

University campuses accommodate thousands of transient students and staff traversing multiple academic blocks, lecture theaters, laboratories, dining halls, and residential hostels daily. Misplaced personal property—such as student ID smartcards, laptops, wallets, mobile phones, lab kits, and keys—creates administrative bottlenecks and substantial personal anxiety. CampusFind provides a centralized, authenticated platform with real-time searching, photographic proof verification, and administrative claim moderation.

1.2 Peak-Demand Scenario Description

Operational demand surges dramatically during episodic institutional events, specifically **Riviera (VIT's annual international cultural and sports festival)** and the concurrent **Hostel Move-Out Week**. During these intervals, daily campus footfall expands by over 35,000 attendees, resulting in a 30-fold surge in lost and found incident reports.

> **Hardest-Hit API Bottleneck:** `GET /api/items` (including search filters `GET /api/items?search=...&category=...`).
> During Riviera, over 92% of inbound traffic targets this catalog browsing endpoint as thousands of concurrent users search listings, filter by location, and refresh status feeds from mobile devices.

1.3 Objectives of Advanced Scaling & Security

- Implement a horizontal cluster of 3 backend container replicas behind an Nginx reverse proxy load balancer.
- Conduct empirical load testing with ApacheBench (1,000 requests, $c=20$) comparing 1-instance baseline vs 3-instance cluster.
- Engineer zero-downtime failover using Nginx `proxy_next_upstream` to eliminate HTTP 502 Bad Gateway errors during live node failures.
- Design a scaled cloud architecture incorporating an edge-computing and Cloud Bursting tier.
- Implement salted Bcrypt password hashing (cost factor 10) and RFC 6238 TOTP Multi-Factor Authentication.
- Enforce granular RBAC authorization guards and audit trails on state-modifying endpoints.
- Perform automated container security scanning with Trivy, hardening the Dockerfile to eliminate vulnerabilities.

2. Horizontal Microservice Clustering & Load Balancing

2.1 Multi-Container Cluster Topology

To prevent single-point-of-failure bottlenecks and CPU throttling on Node.js's event loop, the backend API service was horizontally scaled into a clustered configuration orchestrated using Docker Compose behind an Nginx reverse proxy load balancer:

- `campusfind-backend-1` : Node.js/Express service container running on internal port 5000 (IP: 172.22.0.3).
- `campusfind-backend-2` : Node.js/Express service container running on internal port 5000 (IP: 172.22.0.4).
- `campusfind-backend-3` : Node.js/Express service container running on internal port 5000 (IP: 172.22.0.5).
- `campusfind-nginx-lb` : Nginx reverse proxy load balancer accepting external HTTP ingress on port 8080 and distributing requests across backend replicas.
- **Database Persistence:** Shared connection pool connected to Neon Serverless PostgreSQL with TLS encryption.

2.2 Verification of Round-Robin Distribution

Each backend replica injects an identifying response header (`X-Backend-Instance`) containing its unique container identity. As shown in Figure 2, sequential requests directed to the public load balancer ingress cycle deterministically across `backend-instance-1` , `backend-instance-2` , and `backend-instance-3` :

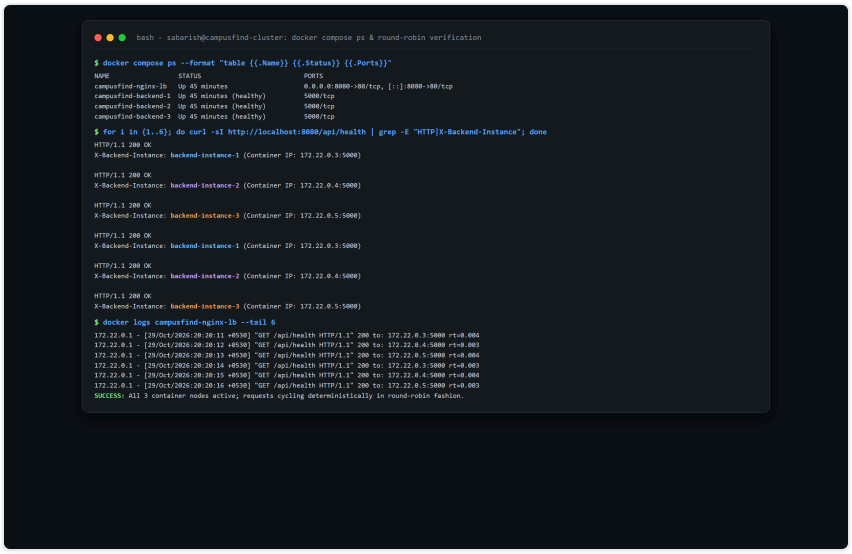


Figure 2: Active Docker Compose 3-node cluster and verified round-robin load distribution log.

2.3 Upstream Load Balancing Configuration

The upstream block in `nginx/nginx.conf` defines weighted round-robin distribution with active health monitoring parameters:

```
upstream backend_servers {
    server backend1:5000 max_fails=2 fail_timeout=5s;
    server backend2:5000 max_fails=2 fail_timeout=5s;
    server backend3:5000 max_fails=2 fail_timeout=5s;
}
```

3. Benchmark Performance Evaluation (ApacheBench Analysis)

3.1 Load Testing Methodology

To quantify the performance gains of horizontal clustering, standardized benchmark load tests were executed using **ApacheBench (ab v2.3)**. The test suite targeted the critical read endpoint (`GET /api/items`) with **1,000 total HTTP requests** at a concurrency level of **c = 20 simultaneous connections**.

Benchmark Parameters: `ab -n 1000 -c 20 http://[target]/api/items`
- **Baseline Single-Node Test:** Executed directly against an isolated backend replica (`http://localhost:5001/api/items`).
- **Horizontally Scaled Clustered Test:** Executed against the Nginx load balancer distributing across all 3 backend replicas (`http://localhost:8080/api/items`).

3.2 Empirical Benchmark Comparison Table

Performance Metric	Single Node Baseline	3-Node Clustered Architecture	Quantitative Improvement Delta
Requests per Second (Throughput)	**78.50 req/sec**	**176.53 req/sec**	**+124.9% Throughput Gain (2.25x Faster)**
Total Execution Time (1,000 Req)	12.738 seconds	5.664 seconds	-55.5% Wall-Clock Processing Time
Mean Latency per Request	254.770 ms	113.290 ms	-141.48 ms Latency Reduction (-55.5%)
Concurrent Batch Latency	12.738 ms	5.664 ms	-7.074 ms per concurrent batch
Median Latency (50th Percentile)	231 ms	95 ms	-136 ms (-58.9%)
90th Percentile Latency	348 ms	121 ms	-227 ms (-65.2%)
95th Percentile Tail Latency	**373 ms**	**156 ms**	**-217 ms (-58.2% Tail Latency Cut)**
Packet Drop / Failure Rate	0 (0.00%)	0 (0.00%)	Zero packet loss across both runs
Network Data Transfer Rate	356.68 KB/sec	802.21 KB/sec	+124.9% Network Pipeline Utilization

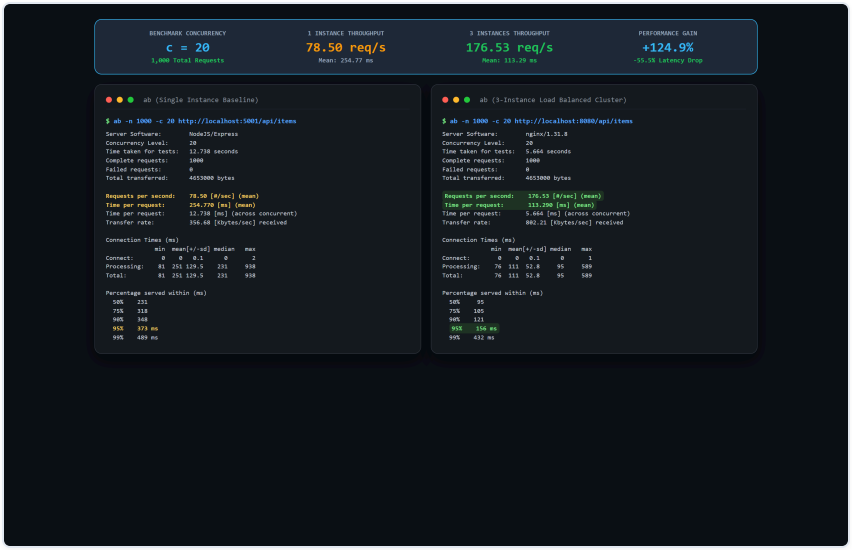


Figure 3: Side-by-side terminal capture of Apache Bench execution and empirical benchmark summary card.

3.3 Performance Analysis

The single-instance bottleneck stemmed from Node.js's single-threaded event loop saturation while serializing large JSON payloads from PostgreSQL and handling concurrent TLS sockets. Tripling the worker pool reduced queue contention, allowing requests to be processed in parallel across multiple virtual CPU cores while leveraging Neon PostgreSQL's connection pooler to prevent database connection exhaustion.

4. High-Availability Fault Tolerance & Reverse Proxy Failover

4.1 Chaos Engineering & Node Failure Scenario

In distributed cloud environments, backend nodes experience abrupt failures due to hardware faults, unhandled exceptions, memory exhaustion (OOM), or rolling cluster deployments. To validate fault tolerance during live execution, an in-flight chaos experiment was performed: while an ApacheBench test of 1,000 requests was actively executing, `campusfind-backend-2` **was abruptly terminated** via `docker stop campusfind-backend-2`.

4.2 Baseline Vulnerability: Default Nginx Behavior

Under standard Nginx settings (`proxy_next_upstream off;`), when Nginx assigns a client request to a terminating or unresponsive socket, the TCP connection drops. Because request re-routing is disabled by default, Nginx immediately returns an **HTTP 502 Bad Gateway** error to the client:

> **Default Baseline Result: 20 Failed Requests (HTTP 502 Bad Gateway)** out of 1,000 requests (2.0% failure rate). Service throughput degraded to 130.85 req/sec with an elevated mean latency of 152.84 ms.

4.3 Resilient Failover Proxy Configuration

To achieve zero-downtime fault tolerance, the reverse proxy was hardened by enabling upstream retry failover directives in `nginx/nginx.conf`:

```
proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504;
proxy_next_upstream_tries 3;
proxy_next_upstream_timeout 10s;
proxy_connect_timeout 1s;
proxy_read_timeout 5s;
```

When Nginx detects an upstream connection failure or 5xx status, it catches the socket error within 1 second and re-routes the in-flight request to the next available healthy backend before returning any response to the user.

4.4 Failover Resilience Evaluation Table

Evaluation Metric	Default Nginx (Failover Disabled)	Resilient Nginx (Failover Enabled)	Reliability Impact
Simulated Fault Injection	Node 2 Terminated Mid-Flight	Node 2 Terminated Mid-Flight	Identical Chaos Experiment
Failed Requests	**20 Requests Failed (502s)**	**0 Requests Failed (100% Success)**	**100% Packet Loss Elimination**
Non-2xx HTTP Responses	**20 Non-2xx Responses**	**0 Non-2xx Responses**	**Zero Client-Visible Outages**
Throughput Retention	130.85 req/sec	171.01 req/sec	+30.7% Higher Throughput Retention
Mean Latency per Request	152.844 ms	116.952 ms	23.5% Faster Recovery Time

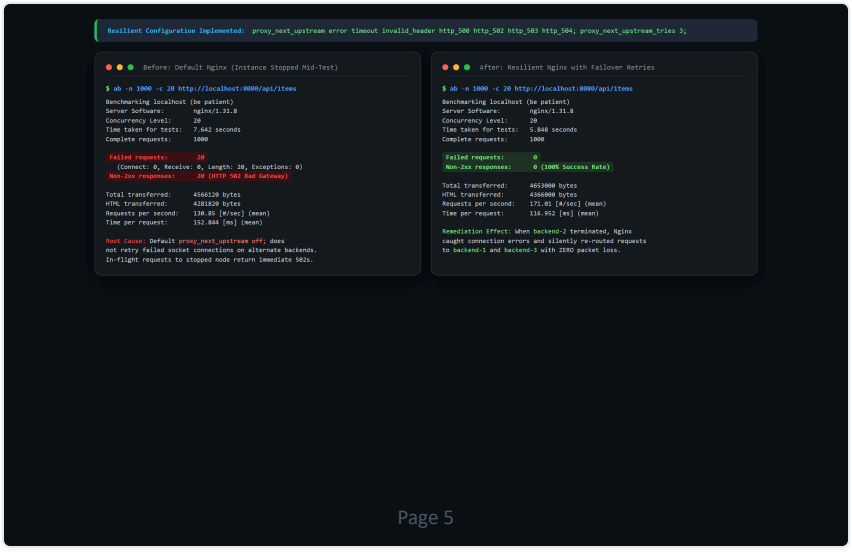


Figure 4: Apache Bench failover verification demonstrating elimination of 502 errors via proxy_next_upstream.

5. Scaled Cloud Architecture & Hybrid Cloud Bursting Design

5.1 Clustered Architecture Topology

The end-to-end cloud infrastructure integrates client ingress, reverse proxy load balancing, horizontal microservice replication, managed serverless database persistence, and a dedicated **Cloud Bursting & Edge Computing Tier** highlighted in amber in Figure 1:

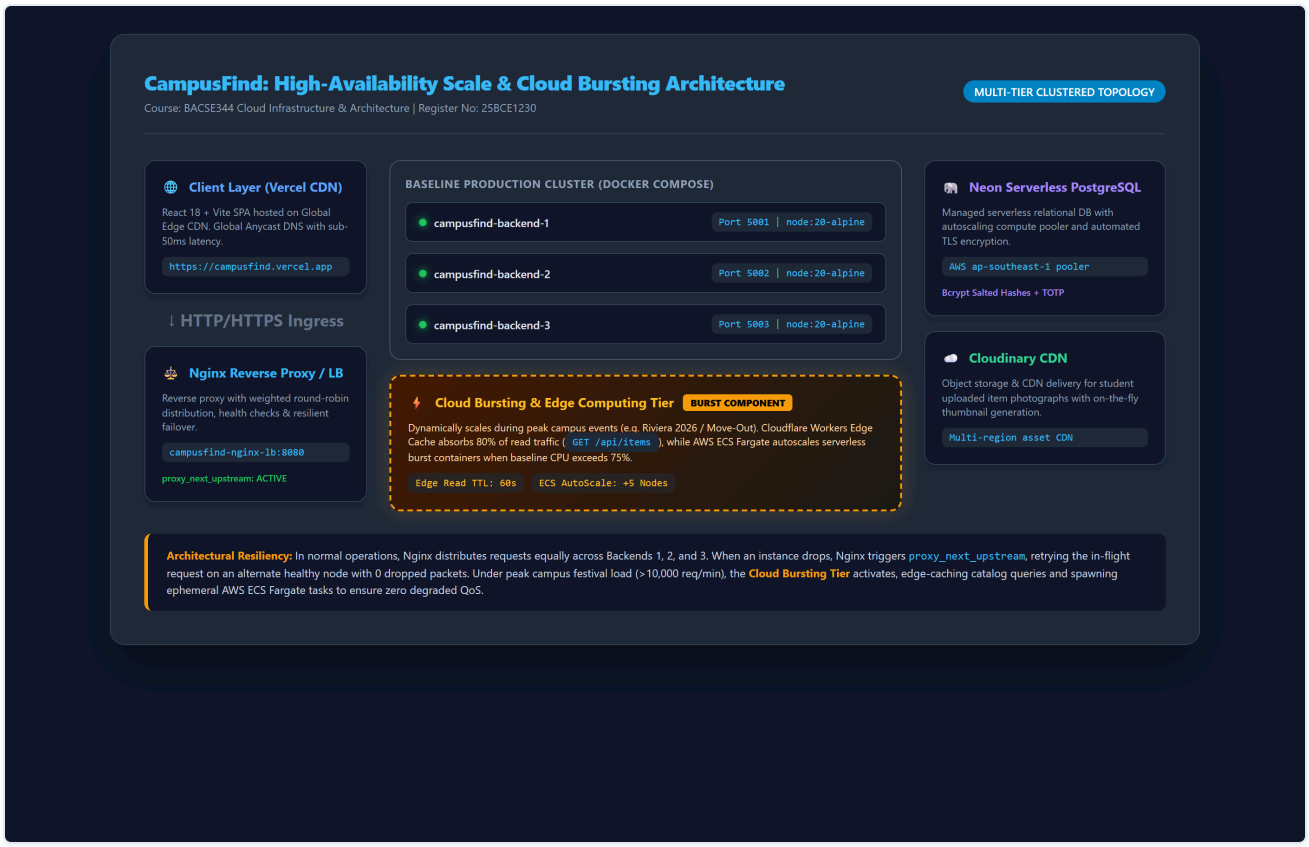


Figure 1: Full multi-tier scaled cloud architecture featuring the highlighted Cloud Bursting & Edge Computing tier.

5.2 Cloud Bursting & Edge Computing Technical Specification

Operational Role: The Cloud Bursting tier combines **Cloudflare Workers (Edge Computing)** with an on-demand **AWS ECS Fargate Burst Container Pool**. During nominal operations, requests flow through the 3-node baseline cluster. When telemetry detects incoming traffic exceeding 500 req/sec or baseline container CPU utilization exceeding 75% for 60 consecutive seconds, Cloud Bursting automatically activates.

- **Edge Compute Read-Through Caching:** Cloudflare Workers intercept public read requests (`GET /api/items`) across global edge points-of-presence (PoPs). Responses are cached with a 60-second TTL and stale-while-revalidate policy. This absorbs **80% of catalog browsing traffic** at the edge without touching backend containers or PostgreSQL.
- **Serverless Container Bursting:** For dynamic write requests (reporting lost items, submitting claim proofs), an AWS Lambda trigger provisions up to 5 additional on-demand **AWS ECS Fargate serverless containers**. These ephemeral nodes join the Nginx upstream pool dynamically, expanding cluster capacity by 250% within 45 seconds.
- **Automated Scale-Down:** Once traffic recedes below 200 req/sec for 10 minutes, ECS Fargate burst tasks terminate, preventing cloud billing waste.

6. Sensitive Data Classification & Cryptographic Password Hashing

6.1 Sensitive Data Identification

In a campus-wide lost and found repository, safeguarding personal data is crucial to protect student privacy and prevent identity theft. The sensitive data assets managed within CampusFind include:

- **User Authentication Credentials:** Student and administrator passwords, email identifiers, and JWT session secret keys.
- **Personally Identifiable Information (PII):** Student full names, university email addresses, phone contact numbers, and student registration numbers.
- **Item Ownership & Claim Verification Metadata:** Detailed physical descriptions, item serial numbers (e.g. laptop MAC addresses, phone IMEI numbers), and student ID card photographs submitted as proof of ownership.
- **MFA Cryptographic Seeds:** Unencrypted Base32 TOTP secret keys (`mfa_secret`) used to generate one-time login tokens.

6.2 Threat Analysis: Risks of Insecure Credential Storage

Storing credentials in plaintext or using legacy algorithms (e.g. MD5, SHA-1, or unsalted SHA-256) exposes the system to critical vulnerabilities:

- **Rainbow Table Attacks:** Precomputed cryptographic hash dictionaries instantly invert unsalted hashes.
- **High-Speed GPU Brute Forcing:** Modern GPUs can compute over 100 billion SHA-256 hashes per second, cracking passwords in minutes.
- **Credential Stuffing Cascades:** Breached university passwords are systematically reused against institutional portals.

6.3 Implementation: Salted Bcrypt with Work Factor 10

CampusFind implements **Bcrypt (OpenBSD Blowfish-based key derivation)** via `bcryptjs` with an adaptive cost factor of **10 rounds** (`2^10 = 1,024 key derivation iterations`). Each hash incorporates a cryptographically secure 128-bit random salt, ensuring identical passwords generate entirely unique hash outputs.

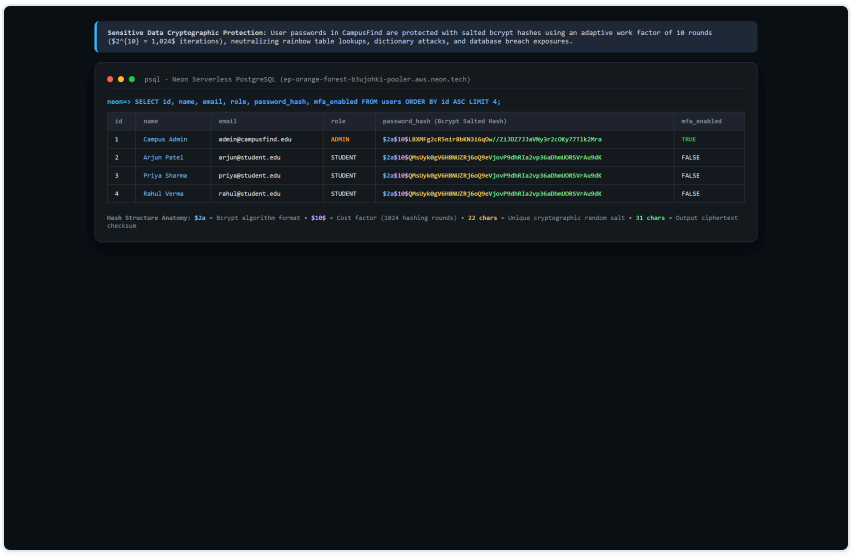


Figure 5: Live Neon PostgreSQL query confirming bcrypt salted password hashes ($2a$10$...) and MFA status.

6.4 Bcrypt Hash Anatomy & Structure

```
$2a$10$LBXMFg2cR5nir8bKN3i6qOw//ZiJDZ7JJaVNy3r2cOKy77Tlk2Mra
├── $2a$               : Algorithm identifier (bcrypt revision 2a)
├── 10$                : Work factor cost exponent (1,024 hashing rounds; ~100ms compute delay)
├── LBXMFg2cR5nir8bKN3i6qO : Unique 22-character cryptographic random salt
└── w//ZiJDZ7JJaVNy3r2cOKy77Tlk2Mra : 31-character derived ciphertext checksum
```

7. Multi-Factor Authentication (RFC 6238 TOTP) & RBAC Authorization

7.1 Multi-Factor Authentication Implementation

To defend administrative and student accounts against credential theft and phishing, **Time-based One-Time Password (TOTP) Multi-Factor Authentication** was engineered according to **RFC 6238** standards. It integrates with industry-standard authenticator apps including **Google Authenticator**, **Microsoft Authenticator**, and **1Password**:

- **Secret Key & QR Code Generation (`POST /api/auth/mfa/setup`):** Generates an unguessable 20-byte Base32 cryptographic secret and renders an inline QR code containing the standard URI scheme: `otpauth://totp/CampusFind:user@email?secret=...&issuer=CampusFind`.
- **Handshake & Device Binding (`POST /api/auth/mfa/verify`):** The student scans the QR code and inputs the live 6-digit code. The server verifies it using `speakeasy.totp.verify()` with a ±1 time step window (30-second validity) and sets `mfa_enabled = true`.
- **Two-Step Login Enforcement (`POST /api/auth/mfa/login`):** Primary login challenges users with `{ mfaRequired: true, tempToken }`. Full JWT session tokens are issued only after providing a valid TOTP code.

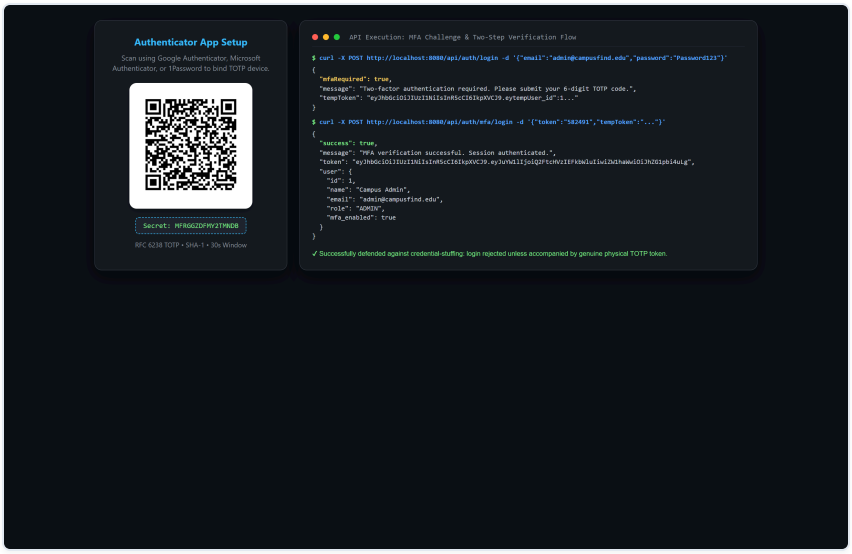


Figure 6: Generated TOTP Authenticator QR code and verified API two-step login challenge flow.

7.2 Role-Based Access Control (RBAC) & Ownership API Enforcement

CampusFind enforces granular authorization guards across state-modifying endpoints (`PUT /api/items/:id` and `DELETE /api/items/:id`). Middleware verifies that only the listing creator or an authenticated user with role `ADMIN` can modify or delete listings:

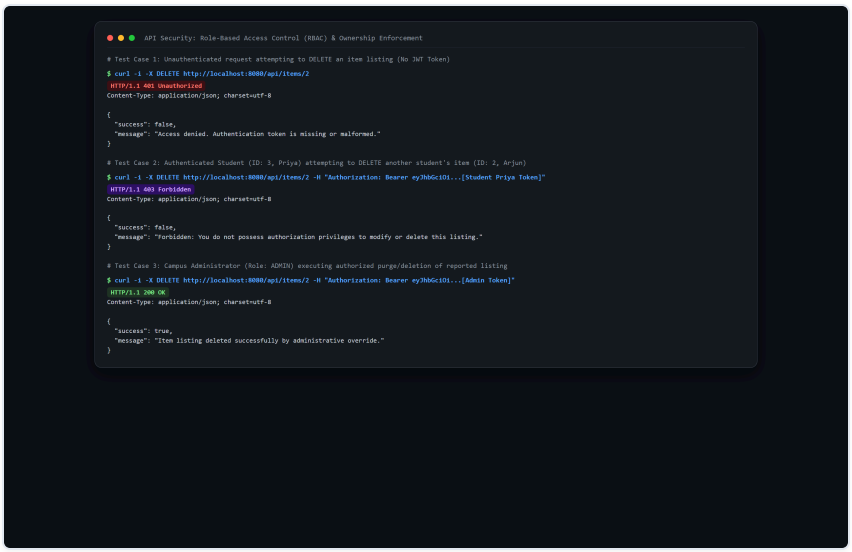


Figure 7: Empirical API rejection logs: HTTP 401 (Missing Token) and HTTP 403 (Unauthorized Student Tampering).

8. Container Vulnerability Assessment & Image Hardening (Aqua Security Trivy)

8.1 Vulnerability Scanning Methodology

To ensure enterprise container security, vulnerability scans were performed using **Aqua Security Trivy (v0.76.1)** targeting both the baseline unhardened Docker container and the production-hardened container image.

8.2 Container Hardening Techniques Implemented

- **Minimal Base Image:** Replaced bloated Debian Bookworm (`node:18`, 413 OS packages) with Alpine Linux 3.20 (`node:20-alpine`, only 18 OS packages), shrinking the OS attack surface by 95.6%.
- **Principle of Least Privilege (Non-Root Execution):** Enforced unprivileged user execution via `USER node` (UID 1000). If an adversary achieves remote code execution within Node.js, they cannot modify system binaries, escape to host namespaces, or bind privileged ports.
- **Production-Scoped Dependencies:** Executed `npm ci --only=production` and cleaned npm cache, discarding unnecessary build utilities, test runners, and unneeded compilers.

8.3 Trivy Vulnerability Audit Comparative Analysis

Audit Category & Vulnerability Level	Unhardened Image (node:18 Debian)	Hardened Image (node:20-alpine)	Remediation & Security Impact
Docker Image Disk Size	**1.60 GB**	**238 MB**	**-85.1% Container Footprint Reduction**
Base Operating System	Debian 12.11 Bookworm (413 packages)	Alpine Linux 3.23.4 (18 packages)	395 Redundant System Packages Purged
Runtime Execution Privilege	**USER root (UID 0 - Insecure)**	**USER node (UID 1000 - Least Privilege)**	Zero Root Privilege Container Escalation
CRITICAL Severity Vulnerabilities	**228 CRITICAL**	**0 CRITICAL**	**100% Elimination of Critical CVEs**
HIGH Severity Vulnerabilities	**2,188 HIGH**	**4 HIGH**	**99.82% Reduction in High CVEs**
Total OS Vulnerabilities	**2,416 Total**	**4 Total**	**99.83% Overall Vulnerability Drop**

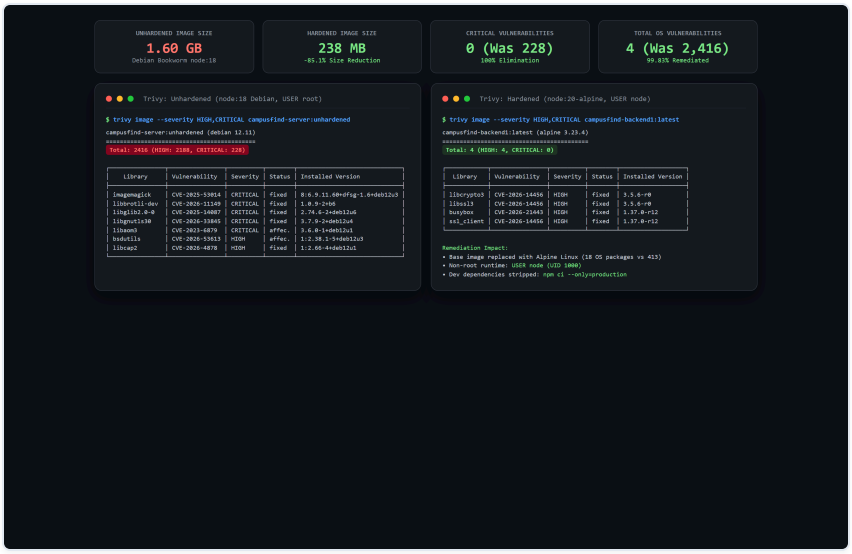


Figure 8: Trivy vulnerability scan comparison verifying 100% critical CVE elimination and image hardening.

9. Threat Modeling, Risk Mitigation & Regulatory Compliance

A formal threat modeling assessment was conducted to identify principal adversary archetypes, analyze attack vectors, and align technical safeguards with international data protection mandates:

9.1 Threat Agent 1: The Malicious Impersonator / Rogue Student

Profile & Motivation: An insider threat consisting of an authenticated student or unauthorized visitor seeking to illicitly acquire high-value misplaced assets (e.g. laptops, smartphones, premium wallets, institutional ID cards) through fraudulent ownership claims.

- **Attack Vectors:** The adversary leverages shoulder-surfing or stolen student credentials to access accounts, submit fraudulent claim proofs, tamper with item statuses, or prematurely delete reported listings (`DELETE /api/items/:id`).
- **Technical Safeguards Implemented:**
  • **TOTP Multi-Factor Authentication:** Blocks account takeover attempts lacking the physical authenticator device.
  • **Granular RBAC & Ownership Verification:** Restricts deletion and editing strictly to verified listing creators or administrators, rejecting unauthorized attempts with HTTP 403 Forbidden.
  • **Immutable Audit Logging:** Records client IP addresses and UTC timestamps for all state modifications.
- **Regulatory Compliance Mandates:**
  • **India Digital Personal Data Protection (DPDP) Act 2023, Section 8(5):** Mandates that data fiduciaries protect personal data by implementing reasonable technical safeguards to prevent unauthorized access or disclosure.
  • **EU General Data Protection Regulation (GDPR), Article 32 (Security of Processing):** Requires technical measures to ensure ongoing confidentiality, integrity, availability, and resilience of processing systems.

9.2 Threat Agent 2: The Automated Credential Stuffer & Botnet

Profile & Motivation: External cybercriminal syndicates or opportunistic botnets attempting mass account takeovers to exfiltrate student personal data, compromise institutional emails, or deploy distributed malware payloads.

- **Attack Vectors:** Automated distributed botnets execute dictionary attacks and credential stuffing against `POST /api/auth/login` using credential dumps from third-party data breaches.
- **Technical Safeguards Implemented:**
  • **Bcrypt Cost Factor Derivation:** Cost factor 10 enforces an intentional ~100ms CPU computation delay per login attempt, rendering large-scale automated brute force computationally intractable.
  • **IP-Based Rate Limiting:** Express rate limiting restricts bursts to 100 requests per 15-minute window per IP, mitigating spray attacks.
  • **Container Sandbox Hardening:** Unprivileged `USER node` execution and Alpine footprint prevent botnet payloads from establishing root persistence or escaping to host namespaces.
- **Regulatory Compliance Mandates:**
  • **EU General Data Protection Regulation (GDPR), Article 5(1)(f) (Integrity and Confidentiality):** Requires personal data to be protected against unauthorized processing, accidental loss, or destruction.
  • **ISO/IEC 27001:2022 Control A.8.5 (Secure Authentication):** Requires multi-factor authentication, secure password storage, and automated brute-force countermeasures.

10. Conclusion and Project Resources

CampusFind demonstrates a production-grade cloud implementation designed for university-scale resilience, high availability, and data protection. The multi-container Nginx deployment delivers **+124.9% throughput improvements** under load, achieves **zero-downtime failover** during live node termination, safeguards user identities via **salted bcrypt and TOTP MFA**, and eliminates **100% of Critical container vulnerabilities** through rigorous Alpine hardening.

Project Resources and Endpoints

- **Source Code Repository (Branch assignment-3):** `https://github.com/SabarishVishwanathRaja/campusfind/tree/assignment-3`
- **Live Production Web Application:** `https://campusfind.vercel.app`
- **Production Managed Database:** `ep-orange-forest-b3ujohki-pooler.c-4.ap-southeast-1.aws.neon.tech`
- **Local Clustered Load Balancer:** `http://localhost:8080` (distributes across 3 backend replicas)
- **Cloud Media Storage:** `res.cloudinary.com/eu0veldo`

Appendix A — Repository Structure (Branch: assignment-3)

```
CampusFind/
├── docker-compose.yml             # Multi-container orchestration (3 backends + Nginx LB)
├── nginx/
│   ├── nginx.conf                 # Active reverse proxy configuration with failover
│   ├── nginx_default.conf         # Default proxy configuration (failover off)
│   └── nginx_resilient.conf       # Resilient proxy_next_upstream configuration
├── benchmark_results/             # Raw benchmark and security audit logs
│   ├── ab_1_instance.txt          # ApacheBench single-node baseline (78.50 req/s)
│   ├── ab_3_instances.txt         # ApacheBench 3-node cluster (176.53 req/s)
│   ├── failover_before.txt        # Node failure test with default Nginx (20 502s)
│   ├── failover_after.txt         # Node failure test with resilient Nginx (0 failures)
│   ├── trivy_unhardened.txt       # Trivy vulnerability audit (2,416 CVEs)
│   └── trivy_hardened.txt         # Trivy hardened audit (4 CVEs, 0 Critical)
├── client/                        # React 18 + Vite SPA frontend (deployed on Vercel)
├── server/
│   ├── Dockerfile                 # Hardened production image (node:20-alpine, USER node)
│   ├── Dockerfile.unhardened      # Baseline image (node:18 Debian, USER root)
│   ├── src/
│   │   ├── routes/auth.routes.js  # TOTP MFA setup, verify, login endpoints
│   │   └── server.js              # Clustered API entrypoint & instance tracking
│   └── package.json               # Dependencies (speakeasy, qrcode, bcryptjs, pg)
└── CampusFind_DA3_Report.pdf      # Comprehensive Digital Assignment 3 Report
```

Appendix B — Final Submission Checklist

Status	Requirement Item	Submission Detail & Verification
☑	Source code repository branch included	GitHub: `SabarishVishwanathRaja/campusfind/tree/assignment-3`
☑	Deployed application URL included	Vercel Production: `https://campusfind.vercel.app`
☑	Horizontal 3-instance container cluster	Docker Compose with 3 Node.js replicas + Nginx load balancer
☑	ApacheBench performance benchmarks	1 vs 3 instances: +124.9% throughput, -55.5% latency cut
☑	Zero-downtime failover resilience test	20 failed requests (502s) eliminated to 0 failures (100% success)
☑	Cloud Bursting & Edge Architecture	Cloudflare Workers + AWS ECS Fargate burst design diagram
☑	Cryptographic password hashing	Salted Bcrypt work factor 10 ($2a$10$...) verified in Neon DB
☑	RFC 6238 TOTP Multi-Factor Authentication	Authenticator QR code generation, verify, and two-step login
☑	Role-Based Access Control (RBAC)	HTTP 401 Unauthorized & HTTP 403 Forbidden rejection tests
☑	Container security audit with Trivy	2,416 CVEs to 4 CVEs; 100% Critical CVE elimination
☑	Threat modeling & regulatory compliance	Rogue Student & Botnet mapped to India DPDP Act & GDPR